Operating System Security

Week 7: Security Enhanced Linux (Part 1)

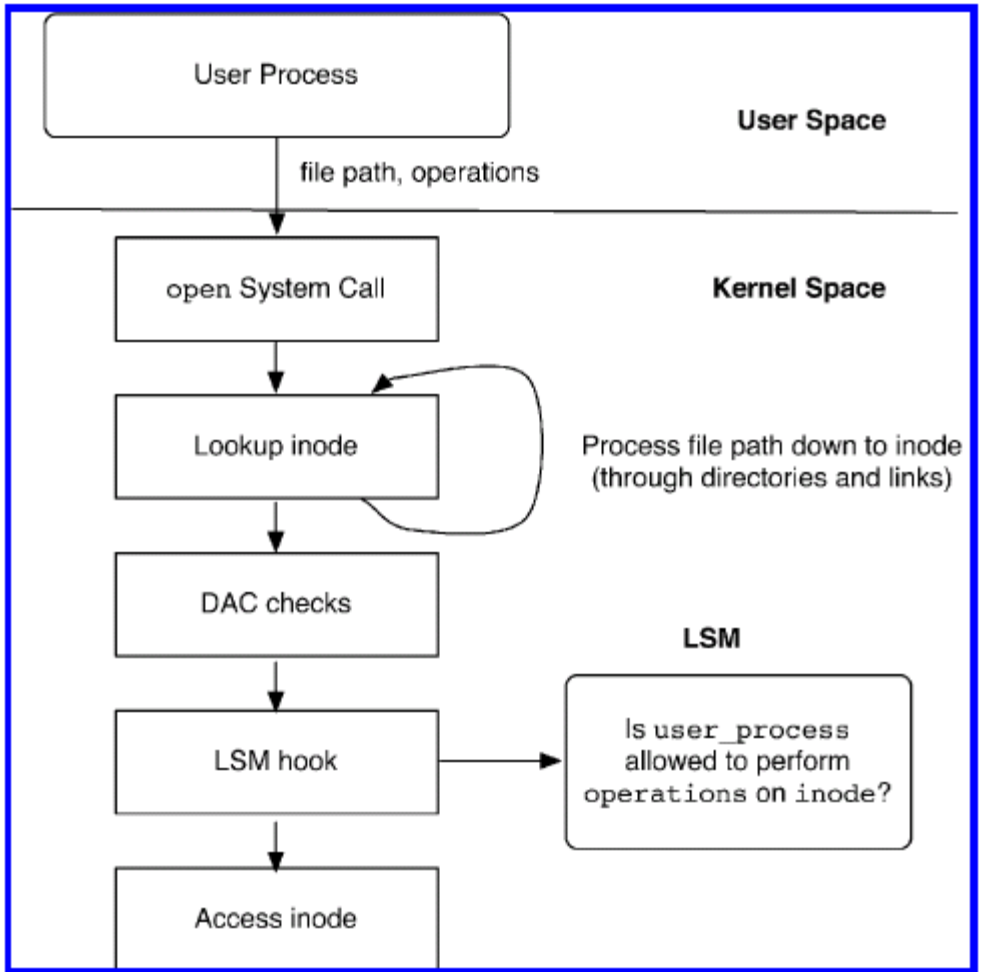
SELinux

MAC → TE

Plugs into the Linux Security Modules Framework.

Provides Mandatory Access Control through Type Enforcement.

You likely don't even realize it, but most Linux distributions (including your Fedora VM) are running SELinux.



What does SELinux do?

It uses a completely different type of access control

- Called "Type Enforcement (TE)"
- Also called "Domain Type Enforcement (DTE)"

In DTE:

- Subjects (processes) are labeled with *domains*
- Objects (e.g., files) are labeled with *types*
  - A label is sometimes called "security context"
- Default deny security posture: explicit rules must be set for a *domain* to have access to a *type*.
  - *Rules can only be added by an administrative user!*
    ** Consider: Is it mandatory or discretionary?

A Peek at SELinux's Object Types

Each object (e.g., file) has a security context associated with it, the most important part of which is the "type".

Example security context (seen by typing "ls –Z"):

-r-------- root root system_u:object_r:shadow_t /etc/shadow

standard Linux *SELinux*

`system_u` is the user (convention: users end with "_u")
`object_r` is a role (convention: roles end with "_r")
`shadow_t` is the type (convention: types end with "_t")

A Peek at SELinux's Subject Domains

ps axZ

Subjects (e.g., processes) run in "domains".

- To see the domains associated with processes, type "ps axZ". Example output (shows httpd is running in domain "httpd_t"):

```
[root@armitage ~]# ps axZ | grep [h]ttpd
unconfined_u:system_r:httpd_t:s0 9448 ?        Ss     0:00 /usr/sbin/httpd
```

- Note that the naming convention for "domains" is "_t"!!
- Unfortunately, sometimes this leads people to call domain labels "types".
- Advice: always consider if it is a process or not: if it's a process, the label is a "domain".

Introduction to Domain Type Enforcement (DTE)

Let's talk about it through the "Type Enforcement" section of this article/comic:

- "Your visual how-to guide for SELinux policy enforcement", by Dan Walsh, 2013, URL: https://opensource.com/business/13/11/selinux-policy-guide (last accessed February 27, 2025).

What is the real-world analogy?

- Dogs and cats can be seen as subjects (processes)
- Food can be seen as an object (e.g., file)
- Rules prevent processes from accessing each other's files.

Why Use SELinux?

Fine-grained access control. Can specify that file B is the only file that application A can access.

- Reduces opportunities for privilege escalation.
- If a process is compromised, attacker cannot use that process to read files in user home directories

SELinux ships with the ability to confine services and daemons so that they are only allowed access that is required for their normal operation.

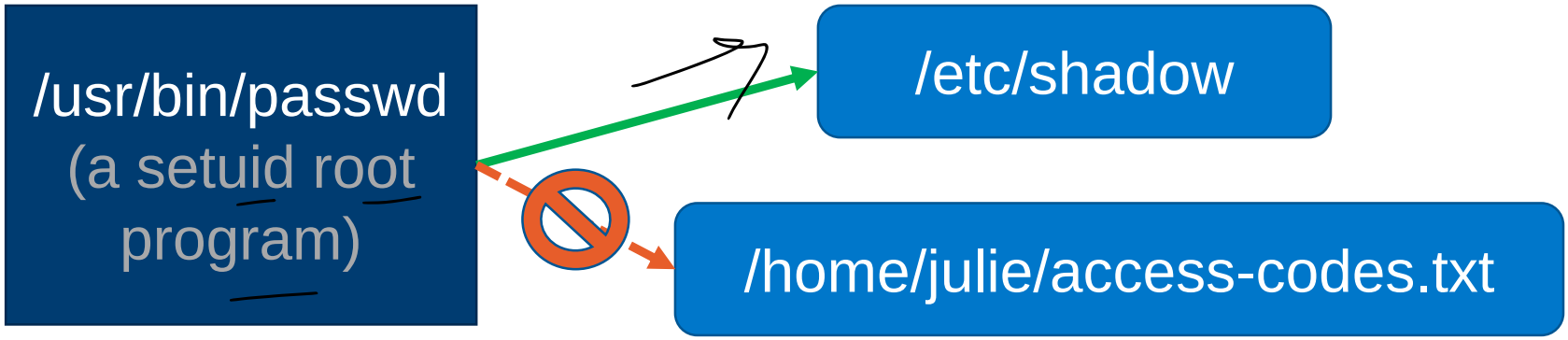
Processes can be set to run "confined"

What does this mean?

- They run in their own domain.

Why is this important?

- This means that the process can only have access to types that they are explicitly configured to (through an access control policy).



Let’s look at how SELinux confines the passwd program

(live demo)

- Set passwd to run
- Look at its domain
- Look at the shadow file’s type
- Should passwd be able to access other root-owned files?

SELinux provides MAC, but Linux is still using DAC underneath

See how when you do an ls –l on an SELinux system, you still see the mode bits

- This is because Linux is still also using DAC
- Note that SELinux can be disabled
- If SELinux is enabled, the DAC rules are checked first, and if they pass, then the SELinux MAC rules are checked.

For evidence of the SELinux system labels (also known as contexts), type "ls –Z" and you will see something like this:

```
-r-------- root root system_u:object_r:shadow_t /etc/shadow
```

standard Linux *SELinux*

Quick summary

Files (or more generally, objects) are labelled with a security context that contains a type.

Processes (or more generally, subjects) are labelled with a security context that contains a domain.

Accesses are specified between domain labels and type labels.

- This differs from Linux where access is specified between users/groups and individual files.

Note: It is also possible to use MCS and MLS policies.

Checking the Status of SELinux on a System

`sestatus` sestatus

- Provides status information about SELinux on system (enabled/disabled, enforced/permissive)
- **Enforced** means it's running and actively blocking denied accesses.
- **Permissive** means it's running but not blocking denied accesses (only logging them).

You can set the enforcement level through:

- `setenforce enforcing`

You can change to "permissive mode" by:

- `setenforce permissive`

Targeted Policy

- The rules for the passwd program already exist by default in a policy called the “targeted policy”.
- So you don’t need to worry about setting up the passwd_t and shadow_t enforcement rules for the passwd file, and many other files and networked services that normally exist on Linux.
- Httpd (web server) is another example of how SELinux confines networked services by default.
- Policies for many standard Linux services (networked and not) exist in this policy.

Targeted Policy Exercise

- You are expected to complete this exercise; the quiz will contain questions.

- Read and complete the steps in Chapter 4.1 and 4.2 of this documentation:
  - "Red Hat Enterprise Linux 6: Security-Enhanced Linux User Guide", https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/6/pdf/security-enhanced_linux/Red_Hat_Enterprise_Linux-6-Security-Enhanced_Linux-en-US.pdf (last accessed Feb. 27, 2023)

  - First see important notes on next slide!!

Targeted Policy Exercise: Setup

1. Install Apache HTTPD on Fedora (as root).
   - Install HTTPD packages.

     ```
     # dnf install httpd -y
     ```
   - Start the HTTPD service.

     ```
     # systemctl start httpd.service
     ```

2. Important note about Chapter 4.2, Procedure 4.3, step 3:
   - you cannot change the context of httpd to unconfined_exec_t anymore – other parts of the policy disallow it. Instead change it to bin_t to get the same result.

   - Another method: put httpd domain in permissive mode to see the same effect (type in root shell "`semanage permissive -a httpd_t`"). Note it takes some time to complete.

More on permissive domains

What is the difference between unconfined processes and processes running in a permissive domain?

To see domains running in permissive mode:

- `semanage permissive -l`

To remove a domain's running in permissive mode:

- `semanage permissive -d <domainname_t>`

Targeted Policy Exercise Take-Away

- Consider: when do we get any benefits from SELinux?
  - Hint: what happened when we changed httpd to run unconfined?

- Is using SELinux ever worse than using Linux without SELinux?

Recall MAC Matrices – E.g., using a military classification scheme

	Unclassified	Confidential	Secret	Top Secret
Unclassified	rwx			
Confidential	rx	rwx		
Secret	rx	rx	rwx	
Top Secret	rx	rx	rx	rwx

Notice how in MAC, accesses are specified between labels!

But note this matrix will always be this size (as the labels are based on a static set of classifications).

SELinux uses MAC (achieved through TE): Can imagine that it looks similar…

	Unconfined_t	**Shadow_t**	**http_content_sys_t**	**Samba_share_t**	**…**
Unconfined_t	Read, write,…	Read, write, …	Read, write, …	Read, write, …	…
Passwd_t		Read, write, …			**…**
Httpd_t			Read, …		**…**
…	**…**	**…**	**…**	**…**	**…**

Important note about the last slide

It is just a visualization to help you imagine what the protection system looks like.

You will never see an access matrix when actually working with SELinux.

SELinux has its own special mechanisms for manipulating the protection system.

How would the protection system get configured?

Normally there is a default policy, e.g., the “targeted policy” that is loaded by default.

System administrators can modify this policy by installing new policy modules or configuring “booleans”.

Only root can install or configure a policy module.

These policy modules read almost like a script, which gets compiled and then loaded/installed.

SELinux Summary Questions

SELinux allows us to specify that a process only has access to the files it needs to operate

- If the process is running with an effective UID of root, how does this help protect the system?

Why does an SELinux system still keep DAC, when the MAC controls are in place?

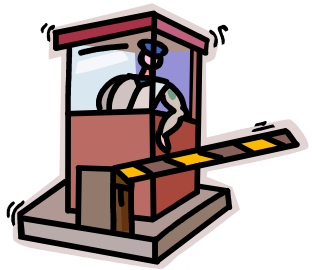
SELinux Security Analysis

Again, we guide the analysis with our system of 7 questions to help understand the security of an operating system

1. Complete mediation: How does the reference monitor interface ensure all security-sensitive operations are mediated correctly?
   - The LSM framework's reference monitor interface is designed to authorize access to the actual objects used by the kernel in security-sensitive operations to prevent vulnerabilities, such as TOCTTOU.

SELinux Security Analysis (con't)

2. Complete mediation: Does the reference monitor interface mediate security sensitive operations on all system resources?
   - The LSM framework mediates operations identified by the LSM community that lead to security-sensitive operations.
   - SELinux reference monitor hooks are stored in: include/linux/security.h of the linux src.
     - Defines a set of function pointers that invoke functions in the LSM.



SELinux Security Analysis (con't -2)

3. Complete mediation: How do we verify that the reference monitor interface provides complete mediation?
   - Source code analysis tools were developed and found bugs in the LSM reference monitor interface, which were subsequently fixed.
   - These tools are an approximation of the complete mediation requirements, and they are not applied on a regular basis.
   - No errors in the reference monitor interface placement have been found since its introduction in Linux 2.6.

SELinux Security Analysis (con't -3)

4. Tamperproof: How does the system protect the reference monitor, including its protection system, from modification?
   - SELinux runs in the kernel
   - Linux has operations that enable access to kernel memory, E.g., the /proc filesystem, but SELinux protection state limits access to trusted processes
   - **Policy can only be changed by the superuser.**

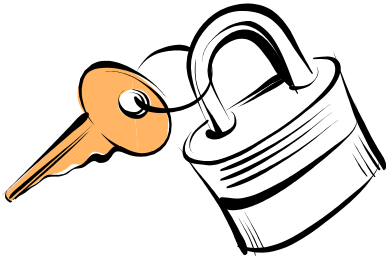
5. Tamperproof: Does the system's protection system protect the TCB programs?
   - Same as Linux without SELinux enabled – could enable ASLR and driver signing, but its often not the default.

SELinux Security Analysis (con't-4)

6. Verifiable: What is the basis for the correctness of the system's TCB?
   - For this large a code base, written mostly in non-type-safe languages, by a variety of developers, verification cannot be complete in practice.

SELinux Security Analysis (con't-5)

7. Verifiable: Does the protection system enforce the system's security goals?
   - SELinux policies define a precise, mandatory specification of the allowed operations in the system.
     - As a result, it is possible to build an information flow representation from the SELinux policies
   - Also, the MLS policy ensures information flow secrecy satisfies the simple-security and *-security properties.



Summary Questions

What are the security strengths of SELinux?

What are the weaknesses of SELinux?

How could SELinux be improved?